

Documentazione progetto TIW – Versione JavaScript

Matteo d'Amato

18 giugno 2026

Indice

1	Introduzione	2
1.1	Architettura generale	2
1.2	Database design	2
2	Application design	3
2.1	Pagina di login	3
2.2	Interfaccia amministratore	4
2.3	Interfaccia responsabile	4
2.4	Interfaccia collaboratore	5
3	Componenti comuni	6
3.1	Connection Handler	6
3.1.1	Configurazione del pool	6
3.1.2	Ciclo di vita	6
3.1.3	Utilizzo nelle servlet	6
3.2	Filtri	7
4	Servlet	7
5	Gestione errori e validazioni	8
6	Componenti CSS principali	9
7	Funzionalità JavaScript	9
7.1	Undo e redo in AdminHome	9
7.2	Switch tab in ManagerHome	10
8	Conclusioni	10

1 Introduzione

In questa documentazione viene descritta la versione JavaScript del progetto. L'attenzione è posta sull'architettura generale, sul database, sulle interfacce e sul comportamento delle servlet.

1.1 Architettura generale

La versione JavaScript dell'applicazione è stata organizzata secondo una classica architettura a livelli, con una netta separazione tra presentazione, logica applicativa e dati. Il client interagisce con il server tramite richieste HTTP asincrone, mentre le servlet costituiscono il punto di ingresso di tutte le operazioni e si occupano di raccogliere i parametri, verificare la correttezza dei dati e coordinare il flusso applicativo. La logica di accesso alla base di dati è centralizzata in classi DAO, che isolano le query SQL dal resto dell'applicazione e rendono più semplice la gestione di entità e relazioni del dominio. L'interfaccia è realizzata come applicazione a singola pagina, così da aggiornare in modo dinamico solo le porzioni di contenuto che effettivamente cambiano, senza ricaricare l'intera pagina. JavaScript gestisce il rendering delle viste, l'interazione lato client e l'aggiornamento parziale dell'interfaccia. Per ridurre la duplicazione del codice e garantire un comportamento uniforme, alcune funzionalità trasversali sono state raccolte in componenti comuni, come il gestore delle connessioni al database e i filtri per l'accesso alle pagine. Questa organizzazione è particolarmente adatta al progetto, perché permette di gestire in modo chiaro i diversi ruoli dell'applicazione, i vincoli sui dati e la necessità di preservare i dati già inseriti lato client anche in caso di errori di salvataggio.

1.2 Database design

Lo schema del database riflette la struttura gerarchica del progetto. Un *progetto* è associato a un solo responsabile e può contenere più *WP*. Ogni *WP* appartiene a un solo progetto e può contenere più *task*. Ogni *task* appartiene a un solo *WP* e può essere assegnato a più *collaboratori*, mentre ogni collaboratore può partecipare a più *task*, anche in *WP* diversi. L'assegnazione di un collaboratore ad un *task* è salvata nella tabella *task_assignee*.

Tutti gli utenti sono salvati nella tabella *user*, che contiene i dati anagrafici, il ruolo (vincolo *ADMINISTRATIVE* o *TECHNICAL*), la password e un'eventuale fotografia dell'utente.

Le ore di lavoro sono modellate con relazioni separate per ore lavorate e ore pianificate. Per ogni *task* sono memorizzate le ore previste mese per mese nella tabella *planned_hours* e le ore lavorate mese per mese da ciascun collaboratore nella tabella *worked_hours*. Le ore del *WP* derivano dall'aggregazione delle ore dei *task* che contiene. Anche la relazione responsabile è modellata con cardinalità one-to-many: ogni progetto ha un solo responsabile, ma un utente tecnico può essere responsabile di più progetti.

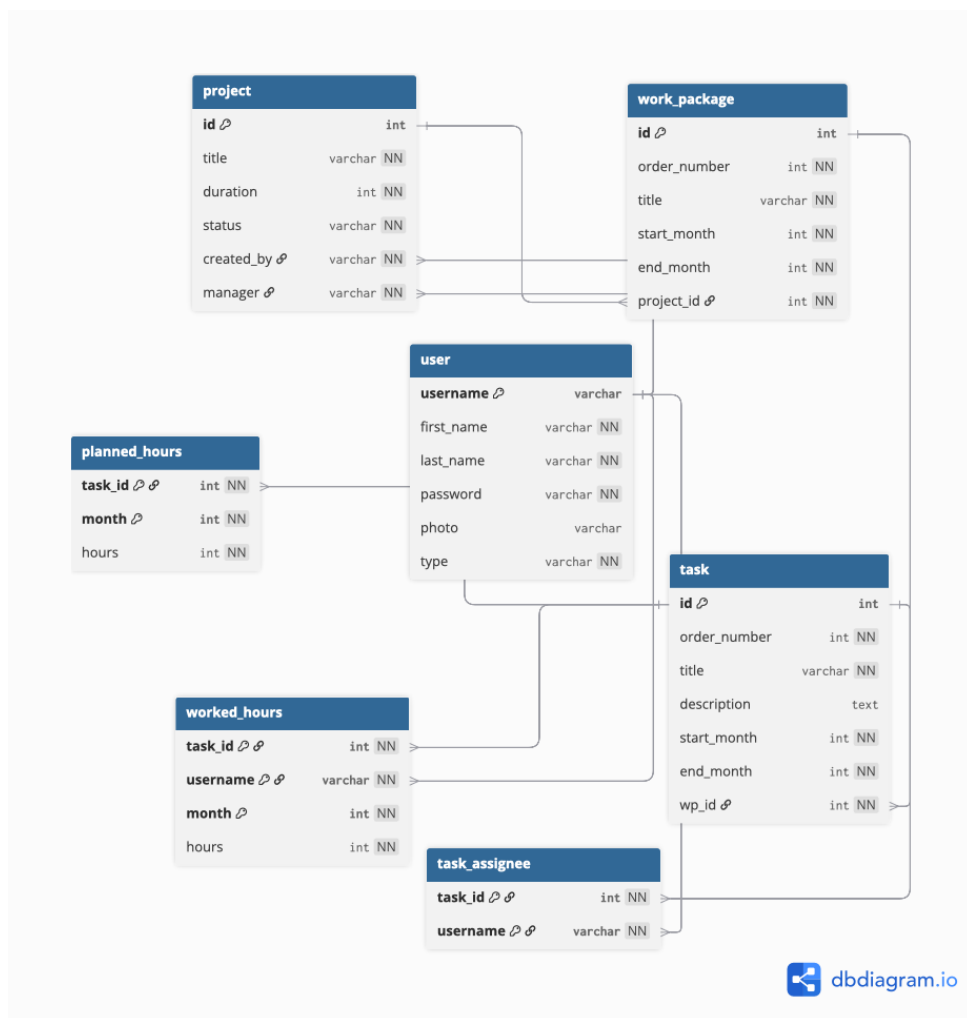


Figura 1: Diagramma del database

2 Application design

In questa sezione vengono analizzate le scelte di design applicativo per ogni pagina della web app. Poiché l'intero diagramma di progetto è di grandi dimensioni, per ogni sezione verrà mostrata solo la parte ad essa riferita. Il diagramma completo può essere trovato nella cartella docs della repository GitHub.

2.1 Pagina di login

Questa schermata rappresenta il punto di accesso iniziale all'applicazione e permette l'autenticazione degli utenti tramite credenziali. Il server provvede a identificare l'utente e, quando necessario, a reindirizzarlo alla pagina corretta. Nel caso in cui l'utente abbia ruolo *TECHNICAL* e sia salvato sia come responsabile di progetto sia come collaboratore, viene reindirizzato alla pagina *Choose Role*. Se invece non è associato a nessun compito, viene reindirizzato alla pagina *No Assignments*. Ogni pagina presenta un bottone *logout* che riporta l'utente alla pagina di login.

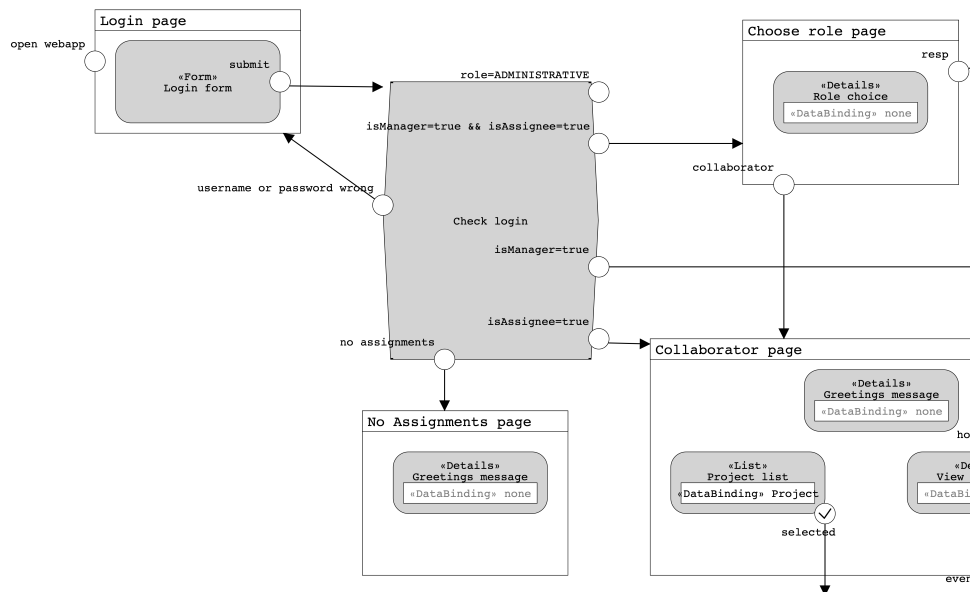


Figura 2: Schema del login

2.2 Interfaccia amministratore

La home dell'amministratore contiene una tabella che permette di selezionare un progetto a sinistra e analizzare ed eventualmente modificare le specifiche a destra.

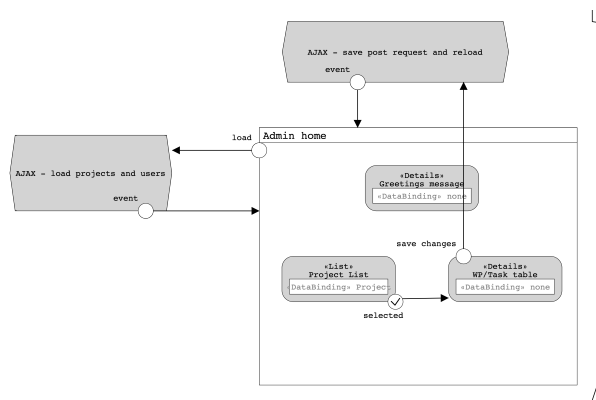


Figura 3: Schema dell'interfaccia amministratore

2.3 Interfaccia responsabile

La home del responsabile contiene tre tab diverse. La prima tab è dedicata all'assegnazione, contenente un form che consente di selezionare progetto, WP e task per assegnare collaboratori e ore previste, oltre a permettere il cambio di stato del progetto da creato ad assegnato. La tab di monitoraggio progetti fornisce una vista aggregata delle ore previste e lavorate per progetto, WP e task, supportando il controllo dell'avanzamento e l'eventuale conclusione del progetto. La tab di monitoraggio collaboratori consente al responsabile di analizzare il lavoro svolto dai collaboratori coinvolti nei propri progetti, con una vista dettagliata delle ore lavorate ogni mese per ogni task.

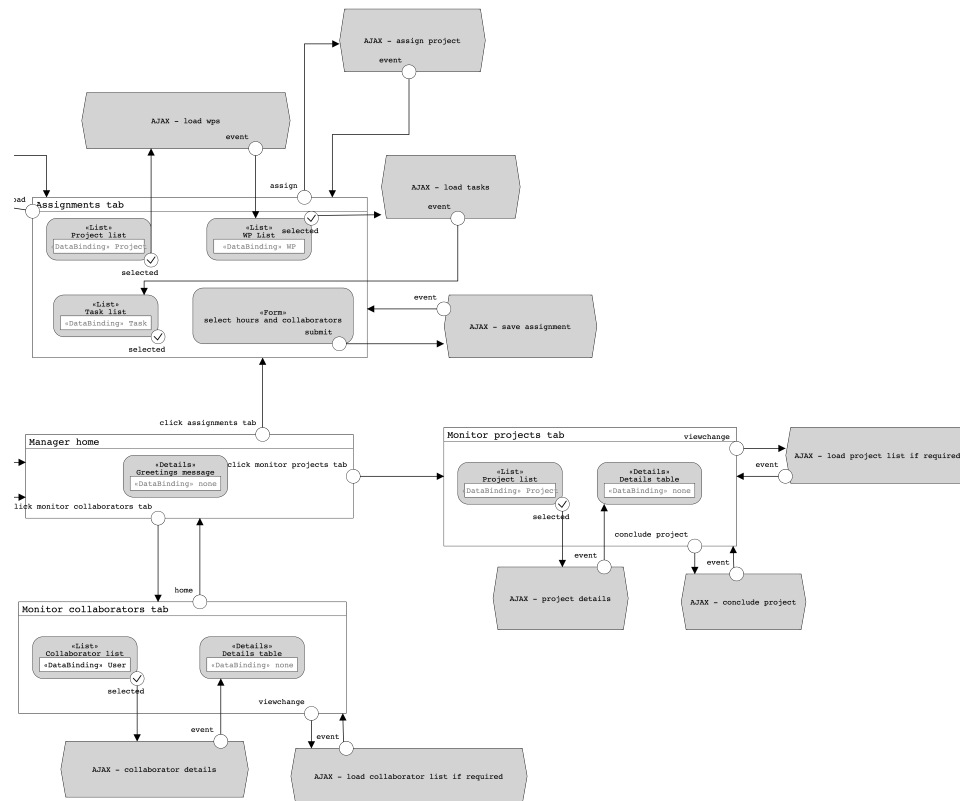


Figura 4: Schema dell'interfaccia responsabile

2.4 Interfaccia collaboratore

La home del collaboratore mostra l'elenco dei progetti ai quali è assegnato. La struttura è simile alle pagine di verifica progetti e monitoraggio progetti, con la lista di progetti sulla sinistra e la tabella con i dettagli, WP e task sulla destra. I task presentano un bottone select, che fa apparire una form per selezionare il mese e inserire le ore lavorate.

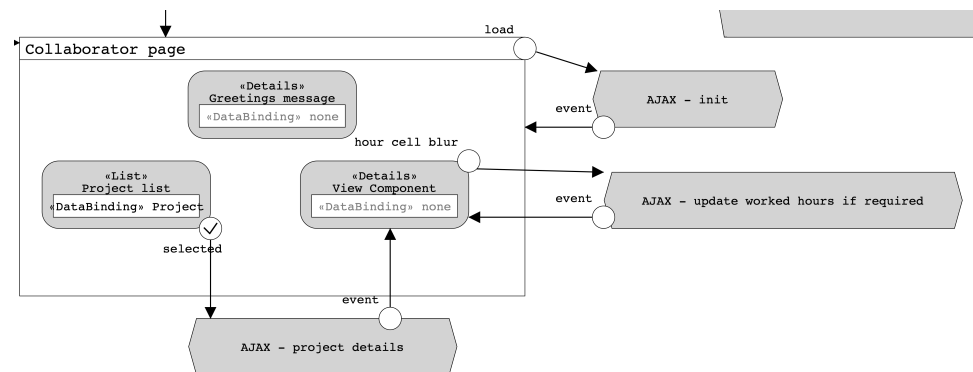


Figura 5: Schema dell'interfaccia collaboratore

3 Componenti comuni

3.1 Connection Handler

La gestione delle connessioni al database è centralizzata tramite **HikariCP**, una libreria Java per il connection pooling ad alte prestazioni. L'obiettivo è evitare che ogni servlet apra e chiuda autonomamente una connessione JDBC, operazione costosa sia in termini di tempo sia di risorse. Con un pool condiviso, le connessioni vengono riutilizzate tra le richieste successive.

3.1.1 Configurazione del pool

Il pool è configurato con i seguenti parametri:

- `maximumPoolSize = 10`: numero massimo di connessioni simultaneamente attive nel pool.
- `minimumIdle = 2`: numero minimo di connessioni mantenute aperte anche in assenza di traffico, per garantire tempi di risposta rapidi.
- `connectionTimeout = 30 000ms`: tempo massimo che una servlet può attendere per ottenere una connessione dal pool prima che venga lanciata un'eccezione.
- `idleTimeout = 600 000ms`: tempo massimo durante il quale una connessione inattiva può rimanere nel pool prima di essere chiusa.

3.1.2 Ciclo di vita

Il `HikariDataSource` viene creato e registrato nel `ServletContext` allo startup dell'applicazione, all'interno di un `ServletContextListener` che implementa il metodo `contextInitialized()`. In questo modo il data source è disponibile globalmente a tutte le servlet, senza essere ricreato ad ogni richiesta.

Alla distruzione del contesto, il metodo `contextDestroyed()` chiude esplicitamente il `HikariDataSource`, rilasciando tutte le connessioni aperte e liberando le risorse associate. Questo garantisce uno shutdown pulito dell'applicazione.

3.1.3 Utilizzo nelle servlet

Le servlet ottengono una connessione al database richiamando il metodo `getConnection()` del *ConnectionHandler*, che restituisce un oggetto di tipo `Connection`. La connessione viene poi utilizzata per eseguire le operazioni necessarie sul database e infine rilasciata esplicitamente tramite il metodo `closeConnection(connection)` del medesimo handler. La gestione avviene con un classico blocco `try/catch/finally`, in modo da garantire la chiusura della connessione anche in caso di eccezioni.

```
Connection connection = null;
try {
    connection = ConnectionHandler.getConnection(getServletContext());
    //operations with connection
} catch(Exception e) {
    //exceptions handling
} finally {
    try {
```

```
        ConnectionHandler.closeConnection(connection);  
    } catch (SQLException ignore) {}  
}
```

Questa soluzione permette di centralizzare l'accesso al database e di mantenere sotto controllo l'apertura e il rilascio delle connessioni, evitando dispersioni di risorse e rendendo il codice delle servlet più pulito.

3.2 Filtri

L'accesso dell'utente ad una qualsiasi pagina è controllato da filtri, che verificano le autorizzazioni dell'utente e impediscono che questi possa effettuare richieste non previste per il suo ruolo.

4 Servlet

Nella versione JavaScript dell'applicazione, le servlet rappresentano il punto di ingresso delle richieste HTTP provenienti dal client e coordinano il flusso tra interfaccia, logica applicativa e accesso ai dati. In generale, una richiesta inviata dal browser viene intercettata dalla servlet corrispondente, che legge gli eventuali parametri presenti nella request e, quando necessario, recupera dalla sessione le informazioni relative all'utente autenticato e al suo ruolo.

In questa versione in particolare è stata effettuata la scelta di gestire tutte le tipologie di richieste riguardanti un'interfaccia da una sola Servlet (AdminHome per l'amministratore, ManagerHome per il responsabile, AssigneeHome per il collaboratore). Questa, per riconoscere l'azione richiesta dal client, legge il parametro *action* sulla request e agisce di conseguenza.

A questo punto la servlet esegue i controlli lato server sui dati ricevuti e invoca uno o più DAO per ottenere o aggiornare le informazioni nella base di dati. Una volta completata l'elaborazione, la servlet prepara i dati necessari alla view e, nel caso della versione JavaScript, restituisce risposte pensate per aggiornamenti asincroni o parziali dell'interfaccia, che vengono poi gestiti dal client senza ricaricare l'intera pagina.

Nel caso di una richiesta **GET**, il compito principale della servlet è recuperare i dati da mostrare e fornire il contenuto richiesto. Nel caso di una richiesta **POST**, invece, la servlet riceve i dati inviati tramite form, li valida, verifica i vincoli applicativi e aggiorna la base di dati; successivamente costruisce la risposta più opportuna, ad esempio restituendo i dati aggiornati oppure un messaggio di errore mantenendo sul client le informazioni già inserite.

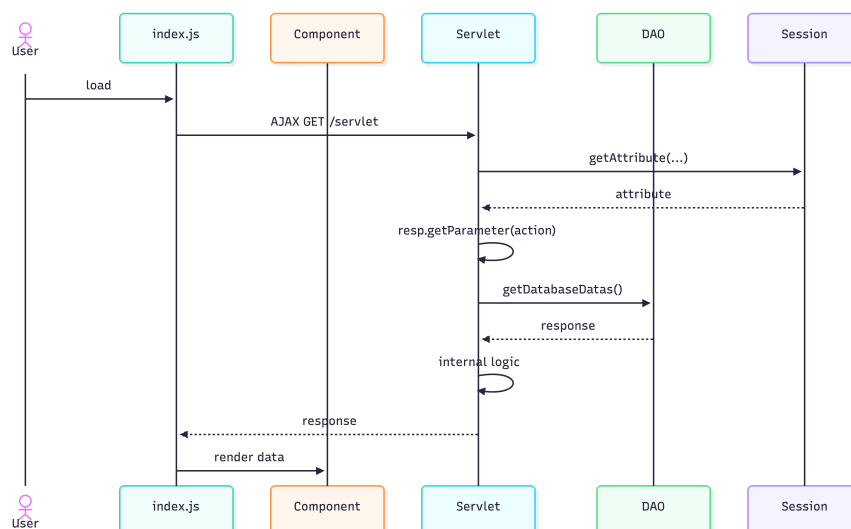


Figura 6: Esempio di richiesta GET

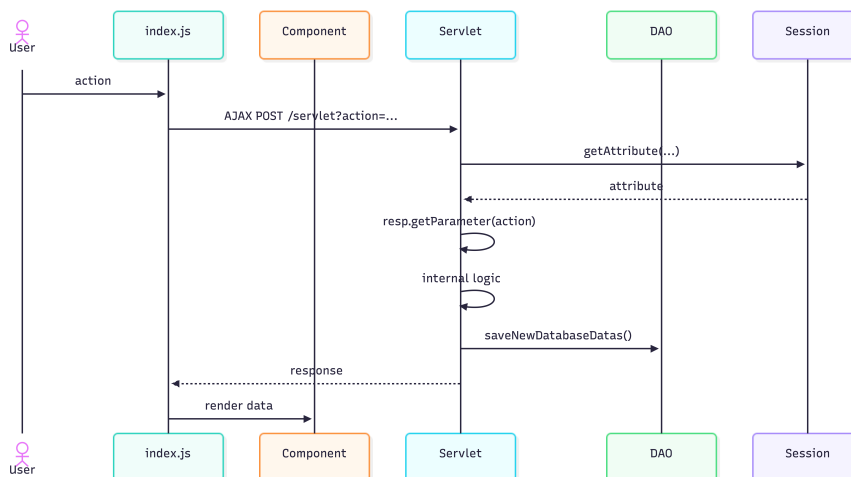


Figura 7: Esempio di richiesta POST

È possibile visualizzare i sequence diagram per ogni servlet nella cartella docs/events della repository GitHub.

5 Gestione errori e validazioni

La validazione dei dati avviene inizialmente lato client, sfruttando le funzionalità native dei form HTML, così da intercettare subito gli errori più semplici prima dell'invio della richiesta. Successivamente, tutti i controlli vengono ripetuti lato server all'interno delle servlet, in modo da garantire la correttezza dei dati anche in caso di richieste manipolate o non valide. Quando una servlet rileva un errore relativo a un form, il messaggio viene inserito in una delle due mappe **missing** o **invalid** a seconda del tipo di errore. Entrambe associano a ciascun campo il rispettivo errore e vengono mandate nel body della response. Inoltre, viene mantenuto lato client lo stato della struttura che l'utente sta costruendo, così da preservare i valori già inseriti in caso di fallimento del salvataggio. Al di fuori dei form, eventuali errori o messaggi di successo

vengono mostrati tramite un alert presente in tutte le pagine, il cui contenuto viene mostrato se la richiesta asincrona ha generato un errore (`!res.ok`).

6 Componenti CSS principali

Componente	Funzione	File principale
site-header, header-inner	Intestazione comune dell'app con logo, saluto e logout.	style.css
main-content, page-heading	Struttura della pagina e titolo principale.	style.css
alert, alert-success, alert-error	Messaggi globali di successo ed errore.	style.css, login.css
form-card, field, btn-primary	Contenitore dei form, campi e bottone principale.	style.css, login.css
login-card	Blocco centrale della pagina di login.	login.css
choose-role-shell, role-card	Sezione di scelta del ruolo con due card.	choose_role.css
manager-home-grid, months-grid	Layout della home del responsabile e griglia dei mesi.	manager_home.css
verify-layout, project-list-panel, project-detail-panel	Layout a due colonne della pagina di verifica e delle viste di monitoraggio.	verify.css

Tabella 1: Componenti CSS principali del progetto

7 Funzionalità JavaScript

7.1 Undo e redo in AdminHome

Il file `admin_home.js`, oltre a presentare la logica di base per inoltrare richieste al server per l'interfaccia amministratore, presenta anche la funzionalità aggiuntiva di poter fare undo e redo sulle modifiche fatte su un progetto. Ciò avviene grazie a uno stack dedicato per ogni azione.

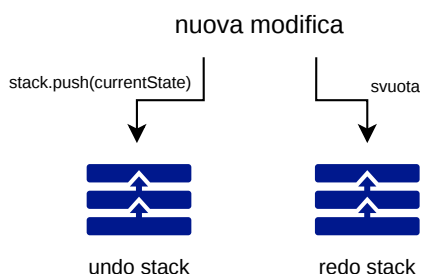


Figura 8: Gestione degli stack per una nuova modifica

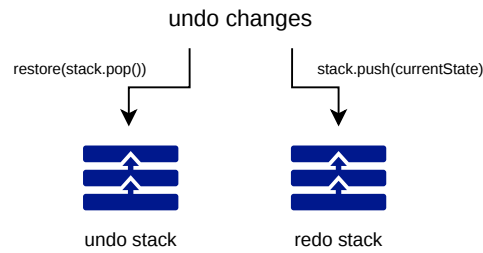


Figura 9: Gestione degli stack per undo

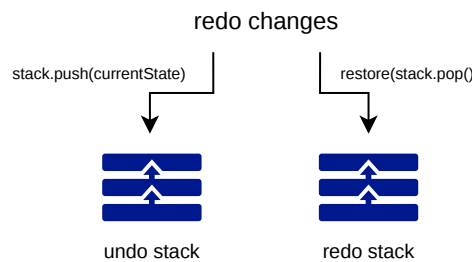


Figura 10: Gestione degli stack per redo

7.2 Switch tab in ManagerHome

Per mantenere il constraint di realizzare tutto in un'unica pagina, nell'interfaccia responsabile ogni pagina è contenuta in una tab che viene caricata dinamicamente dal file `manager_home.js`. Le richieste asincrone al server vengono effettuate solo se necessario (ad esempio nuova dopo modifica), in caso contrario vengono semplicemente nascoste le sezioni che non devono essere mostrate.

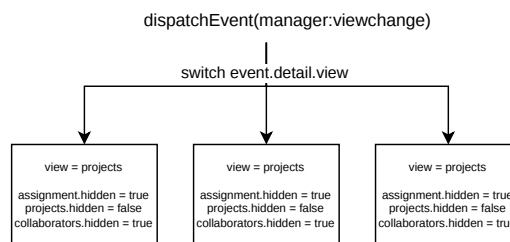


Figura 11: Gestione delle tab

8 Conclusioni

In sintesi, l'architettura adottata separa chiaramente logica applicativa, persistenza e presentazione, rendendo il progetto più ordinato, dinamico e facilmente manutenibile.